

Modul Latihan 1 Advanced Kotlin dan Coroutines

Persiapan

1. Buka **IntelliJ IDEA** atau **Android Studio**.
2. Buat file Kotlin baru (misal: `PraktikumCoroutines.kt`).
3. Pastikan library `kotlinx-coroutines-core` sudah ditambahkan.

1 Pemanasan dan Null Safety

Tujuan untuk membiasakan diri dengan penanganan nilai `null` agar aplikasi tidak crash.

Soal

Salin kode berikut dan lengkapi bagian `// TUGAS 1`:

Kotlin

```
data class Mahasiswa(val nama: String, val email: String?)

fun main() {
    val daftarMhs = listOf(
        Mahasiswa("Budi", "budi@gmail.com"),
        Mahasiswa("Siti", null), // Email kosong
        Mahasiswa("Andi", "andi@yahoo.com")
    )

    println("--- DAFTAR EMAIL MAHASISWA ---")
    for (mhs in daftarMhs) {
        // TUGAS 1: Ambil panjang karakter email.
        // Jika email null, anggap panjangnya 0.
        // HINT: Gunakan Safe Call (?.) dan Elvis Operator (?:)

        val panjangEmail = 0 // <--- GANTI BARIS INI DENGAN KODE ANDA

        println("${mhs.nama} : Panjang email = $panjangEmail")
    }
}
```

Jawaban yang Diharapkan

Output harus menampilkan angka `0` untuk Siti, dan angka panjang karakter untuk Budi dan Andi tanpa error.

2 Coroutines (Serial vs Concurrent)

Tujuan untuk membuktikan bahwa `async` membuat proses loading data lebih cepat.

Soal

Salin kode berikut. Perhatikan bahwa kode saat ini berjalan lambat (Total ± 2 detik). Tugas Anda membuatnya berjalan cepat (± 1 detik).

```
Kotlin
import kotlinx.coroutines.*
import kotlin.system.measureTimeMillis

// Simulasi API call (JANGAN DIUBAH)
suspend fun getProfil(): String {
    delay(1000) // Pura-pura loading 1 detik
    return "Profil: Budi Santoso"
}

suspend fun getTransaksi(): String {
    delay(1000) // Pura-pura loading 1 detik
    return "Transaksi: [Beli Laptop, Beli Mouse]"
}

fun main() = runBlocking {
    println("--- MULAI DOWNLOAD DATA ---")

    val waktu = measureTimeMillis {
        // TUGAS 2: Ubah kode di bawah ini agar berjalan PARALEL (Bersamaan)
        // Gunakan `async` dan `await`

        // --- UBAH BAGIAN INI ---
        val profil = getProfil()
        val transaksi = getTransaksi()
        // -----

        println("Hasil: $profil dan $transaksi")
    }

    println("Total Waktu: ${waktu}ms")
}
```

3: Kotlin Flow (Stream Data)

Tujuan untuk memproses aliran data sensor secara real-time.

Soal

Kita punya sensor suhu yang mengirim data terus menerus. Kita hanya ingin menampilkan peringatan jika suhu **di atas 35°C**.

```
Kotlin
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

// Simulasi Sensor (Producer)
fun sensorSuhu(): Flow<Int> = flow {
    val dataSuhu = listOf(28, 30, 36, 29, 40, 32, 45)
    for (suhu in dataSuhu) {
        delay(500) // Sensor update tiap 0.5 detik
        emit(suhu) // Kirim data
    }
}

fun main() = runBlocking {
    println("--- MONITOR SUHU AKTIF ---")

    sensorSuhu()
        // TUGAS 3a: Filter data. Hanya loloskan suhu > 35
        // .filter { ... }

        // TUGAS 3b: Ubah format data jadi String "PERINGATAN! Suhu: X derajat"
        // .map { ... }

        .collect { pesan ->
            println(pesan)
        }
}
```